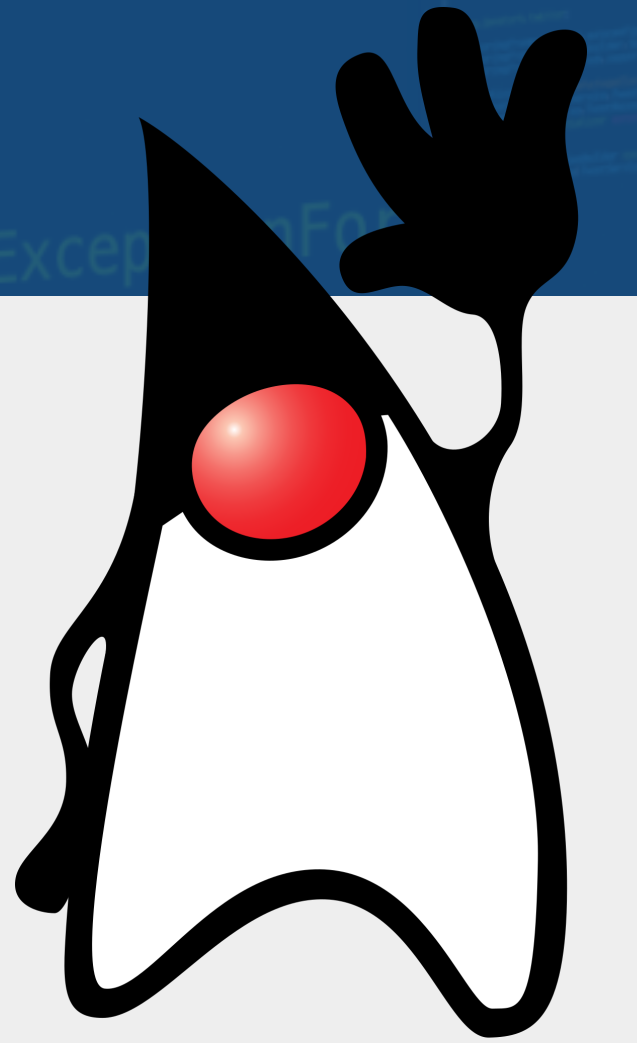


Java Programlama Dili

25. Bölüm

Sırada Ne Var?

Akın Kaldıroğlu



Amaç



- Bu bölümün amacı, bu eğitimi bitirdikten sonra Java'yı öğrenmeye nasıl devam edebileceğinizi ele almaktır
- Dolayısıyla bu bölüm, daha öncekiler gibi değildir, yeni bir konu ele almaz

Neler Öğrendik?



- Java geliştirme ortamı: JDK, komut satırı ve IDE
- Java'nın tip sistemi: ilkel tipler ve referanslar, değişkenler
- İşlemciler ve akış kontrolü
- Nesne: sınıf, durum, davranış, kurucu, this, static, başlatma
- Bellek yönetimi, kod organizasyonu, classpath
- Sarmalama ve bilgi saklama, enum

Neler Öğrenmeliyiz?



- Java'da daha yetkin hale gelmek için şunları öğrenmeye devam etmeliyiz:
- Nesne merkezli programlamanın (OOP) daha ileri özellikleri
- Sıra dışı durum yönetimi, torbalar/koleksiyonlar ve jenerikler gibi tamamlayıcı konular
- Fonksiyonel programlama
- Multi-threaded programlama ve eşzamanlılık (concurrency)

Nesne Merkezli Programlama - I



- Nesne merkezli programlamayı, bu eğitimde ele alınmayan aşağıdaki konularla ilerletmeniz gereklidir:
- **Kalıtım (inheritance)**: bir sınıfın başka bir sınıfın üyelerini devralması; extends
- **Çok biçimlilik (polymorphism)**: aynı çağrının, nesnenin gerçek tipine göre farklı davranması
- **Soyut metotlar ve sınıflar (abstract method and class)**: gövdesi olmayan metotlar ve bunları tamamlayan alt sınıflar

Nesne Merkezli Programlama - II



- **Arayüzler (interface)**: davranış sözleşmesi; bir sınıf birden fazla arayüzü uygulayabilir
- **Mühürlü sınıflar (sealed classes) ve örüntü eşleme (pattern matching)**: bir tip hiyerarşisini sınırlamak ve tipe göre dallanmak
- Bu konular birlikte öğrenilir çünkü birbirleriyle bağlantılıdır

Değişmezlik ve record



- **Değişmezlik (immutability):**
 - Sarmalama, nesnenin durumunu dışarıdan korur; değişmezlik ise durumun hiç değişmemesini sağlar
 - **record:** yalnızca veri taşıyan değişmez sınıflar
- Bu şekilde Java'daki nesne merkezli programlama mekanizmalarını büyük ölçüde tamamlamış olursunuz

Tamamlayıcı Konular



- **İstisnalar, sıra dışı durumlar (exceptions):** hata durumlarını try-catch ile yönetmek, throws ile bildirmek
- **Torbalar/koleksiyonlar (collections):** `List`, `Set`, `Map` — dizilerden esnek veri yapıları
- **Jenerikler (generics):** `List<String>` gibi tip güvenli torbalar
- Bu üç konu, günlük Java işlerinizin neredeyse tamamında birlikte kullanılır
- Aynı zamanda bu konular Java'da nesne yapılarının birer güzel uygulamasıdır

Fonksiyonel Programlama



- **Lambda ifadeleri (lambda expressions):** bir metodun gövdesini değer gibi kodda gezdirmek
- **Fonksiyonel arayüzler (functional interface):** tek soyut metodu olan arayüzler
- **Metot referansları (method reference):** `String::length`
- **Stream ile veri işleme:** koleksiyonlar üzerinde filtreleme, dönüştürme ve toplama
- Fonksiyonel programlama, nesne merkezli programlamanın yerine geçmez; onunla birlikte kullanılır

Çok Kanallı Programlama - Multi-threading



- **Kanallar, iş parçacıkları (threads):** aynı programda birden fazla işin birlikte yürümesi
 - Buna **multi-threaded programming** (çok kanallı programlama) de denir
- **Sanal iş parçacıkları (virtual threads):** Java 21'den beri, çok sayıda iş parçacığını ucuza çalıştırmak
- **Eşzamanlılık (concurrency):** Çok kanallı ortamda verinin ve nesnenin durumunun tutarsız hale gelmesinin önüne geçmek

İhtiyaca Göre



- Aşağıdaki konular da ihtiyaca göre öğrenilebilir:
- **Giriş-Çıkış (I/O, Input/Output):** Dosyaya yazmak ve dosyadan okumak (file I/O) vb. JVM ile dış dünya arasındaki veri alış-verişi
- **JDBC (Java Database Connectivity):** İlişkisel veri tabanları ile iletişim, veri, nesnelerin durumlarını yazmak ve okumak
- **Reflection:** JVM'e yüklenmiş tipleri incelemek

Sonraki Adımlar



- Bu konularla birlikte Temel Java'da (Java SE) iyi bir noktaya gelmiş olursunuz
- Sonrasında REST web servisleri ile Jakarta EE'ye (eskiden Enterprise Java) geçebilirsiniz
- Spring ve JPA gibi frameworkleri öğrenmeye başlayabilirsiniz

Eğitim Sonu Soru ve Cevap Zamanı!

